Beyond the Chatbot

Building LLMs into Business Systems

Matt Hergott
mirabella-ai.com

Presented to Denver Data Dialogues
March 10, 2026

What We'll Cover

01 **Coding Agents**
AI collaborators in the terminal

02 **Structured Outputs**
From conversation to contract — defining what LLMs deliver

03 **Chain-of-Thought**
Orchestrated reasoning

04 **Program Recipes**
Portable blueprints — when the prompt becomes the product

01

Coding Agents

AI collaborators that draft, debug, and iterate alongside you

Major AI Coding Agents Today

Claude Code

Anthropic

Agentic coding in your terminal. Full codebase access.

Codex (OpenAI)

OpenAI

Cloud-based. Parallel tasks in sandboxed environments.

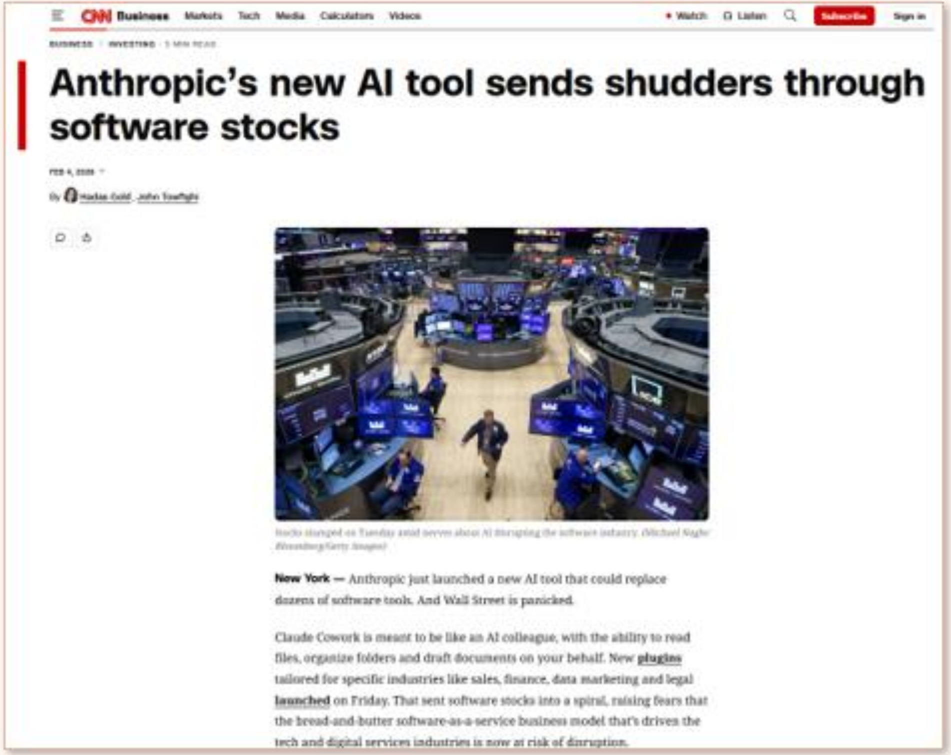
Claude Cowork

Anthropic (new)

GUI-based. Automates file & task management.

Try many. Outages happen.

In the News



Business Markets Tech Media Calculators Videos Watch Listen Subscribe Sign in

Anthropic's new AI tool sends shudders through software stocks

New York — Anthropic just launched a new AI tool that could replace dozens of software tools. And Wall Street is panicked.

Claude Cowork is meant to be like an AI colleague, with the ability to read files, organize folders and draft documents on your behalf. New **plugins** tailored for specific industries like sales, finance, data marketing and legal **launched** on Friday. That sent software stocks into a spiral, raising fears that the bread-and-butter software-as-a-service business model that's driven the tech and digital services industries is now at risk of disruption.

CNN · Feb 4, 2026

AI Tool Triggers Software Stock Bloodbath

[CNN Business, Feb. 4, 2026]

02

Structured Outputs

Teaching LLMs to deliver validated, typed data — not just text

The Problem: Unstructured Information

RAW ER INTAKE TRANSCRIPT

so yeah the patient is um a woman I think she said she's like 47 or maybe 48 and she came in saying her stomach has been really hurting for the past couple days like really bad cramping and she said she threw up twice this morning ... and she said the pain is mostly on her right side kind of low and it gets worse when she moves around or coughs and she has a slight fever the thermometer said 101 point something

?

How do you extract actionable data from this?

- ✕ No database storage
- ✕ No automated routing
- ✕ No urgency scoring

LLM Chat Can Clean It Up... Sort Of

RAW TRANSCRIPT

um ok so this guy came in hes I think 34 and hes saying his chest is really tight and hes been sweating a lot and also having trouble catching his breath and it started about two hours ago he said and he looks pretty anxious...

LLM CHATBOT

Male, approx. 34 y/o, self-ambulatory at arrival. Reports chest tightness, diaphoresis, and SOB onset ~2hr ago. Appears visibly anxious; skin moist/diaphoretic. No stated cardiac hx. Symptoms described as ongoing.

Now what?

Still text. Can't route, store, or automate.

Impressive conversion — but the output is still an opaque blob. Tough to act on it automatically.

Structured Outputs: A Simple Schema

Pydantic (Python)

```python
class MedicalConversion(BaseModel):
    medical_text: str = Field(
      description="Rewrite the
      intake transcript as
      concise clinical notes
      using standard medical
      terminology"
    )
```

JSON Schema (What the API Sees)

```json
{
  "title": "MedicalConversion",
  "type": "object",
  "properties": {
    "medical_text": {
      "type": "string",
      "description":
        "Rewrite..."
    }
  }
}
```

Same output — but now **validated, typed, and inside a data structure**

Add One Field. Change Everything.

```python
class MedicalConversion(BaseModel):
    """Convert patient intake to
medical language."""

    medical_text: str = Field(
      description="...")
    urgency_score: int = Field(
      description="Rate 1-10 based on
      the totality of the intake:
      symptoms, urgency cues,
      and what was not said"
    )
```

A quantum leap in intelligence

One integer field. The model evaluates what was said, how it was said, and what was *not* said.

Traditional NLP misses context, nuance, and implication.

Under the Hood: How It Actually Works

PATH A: CONSTRAINED DECODING

Generate Token → Schema Check → Guaranteed Valid

Every token constrained at model level — output is **guaranteed valid**

PATH B: CODE ENFORCEMENT

Generate → Validate → Retry if Invalid → Return Valid

Schema in prompt. **Retry with error feedback** until valid JSON.

Case Study: ER Triage

Triage One word: what ER doc should do

RiskCalc Worst case + time horizon?

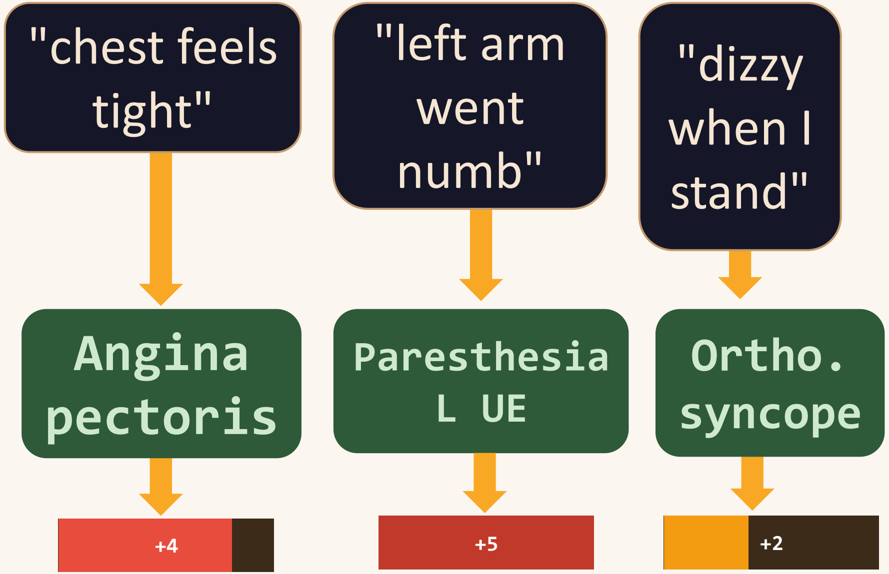
SymptomCluster Group symptoms: acute? pattern?

SymptomRead What did the patient say, and what does it mean?

Layer 1: SymptomRead

Patient language → structured clinical signal

```
class SymptomRead(BaseModel):
  """Patient-reported →
     clinical language."""
  clinical: str = Field(
    description="Medical
      terminology.")
  severity: int = Field(
    description="-5=benign.
      0=unclear. +5=acute.")
```



Each card = one **SymptomRead** object

Layer 2: ClinicalPicture

Pattern recognition across symptoms

```python
class SymptomCluster(BaseModel):
    """Group related symptoms &
       flag acute clusters."""
    symptoms: list[SymptomRead]
    acute: bool = Field(
      description="True if
        requires immediate
        intervention. False
        if stable/non-urgent.")
```

Step 1

New

Cardiac Cluster

⚠ acute: True

- Angina pectoris +4
- Paresthesia L UE +5

symptoms cluster → **MI presentation** → acute: True

Hemodynamic Cluster

✓ acute: False

- Ortho. syncope +2

Isolated, moderate → acute: False ✓

Layer 3: RiskCalc

Worst-case diagnosis + how fast the window is closing

Step 2

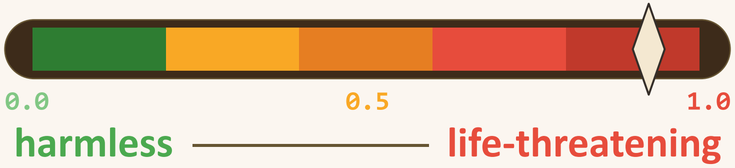
New

```
class RiskCalc(BaseModel):
  """Score overall danger across
      all symptom clusters."""

  clusters: list[SymptomCluster]

  risk: float = Field(
    description="0.0=implausible
      or harmless. 1.0=near-certain
      and life-threatening. Weight
      plausibility × consequence.")
```

Cardiac
⚠ ACUTE

Hemodynamic
✓ stable

risk = 0.92



risk: 0.92

Near-certain & life-threatening

Layer 4: Triage

One word, carrying everything beneath it

```python
TriageRouting = Literal[" surgery",
  "observation", "discharge", …]

class Triage(BaseModel):
  """Final routing judgment."""

  calculations: list[RiskCalc]

  judgment: TriageRouting = Field(
    description="One routing word
    derived from the full chain:
    symptoms → clusters → risks
    → this final decision.")
```

Step 3

New

Angina | Pares. | Syncope

▼

Cardiac ⚠ | Hemodynamic ✓

▼

risk: 0.92 ← *near-certain*

▼

judgment: "surgery"

→ Coronary Artery Bypass Graft

3 symptoms → 2 clusters → 0.92 risk → **one word**

Why Not Just Ask Directly?

Four reasons to chain structured steps instead of

Better thinking	Models reason better when you give them a staircase, not a cliff
Small models, big jobs	Break the problem down and suddenly a cheap model can do it
Show your work	Every intermediate step is a receipt you can audit later
You're the architect	Without structure, the model freelances its logic

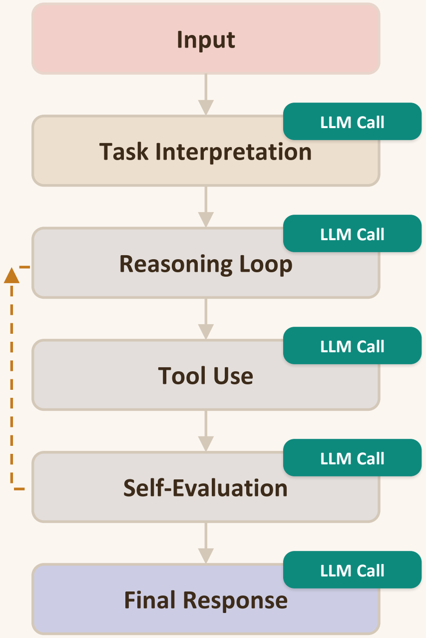
03

Chain-of-Thought in Practice

How structured outputs power reasoning

CoT: An Adaptive Reasoning Loop

Code orchestrates coherent reasoning



KEY CONCEPTS

What is Chain of Thought?

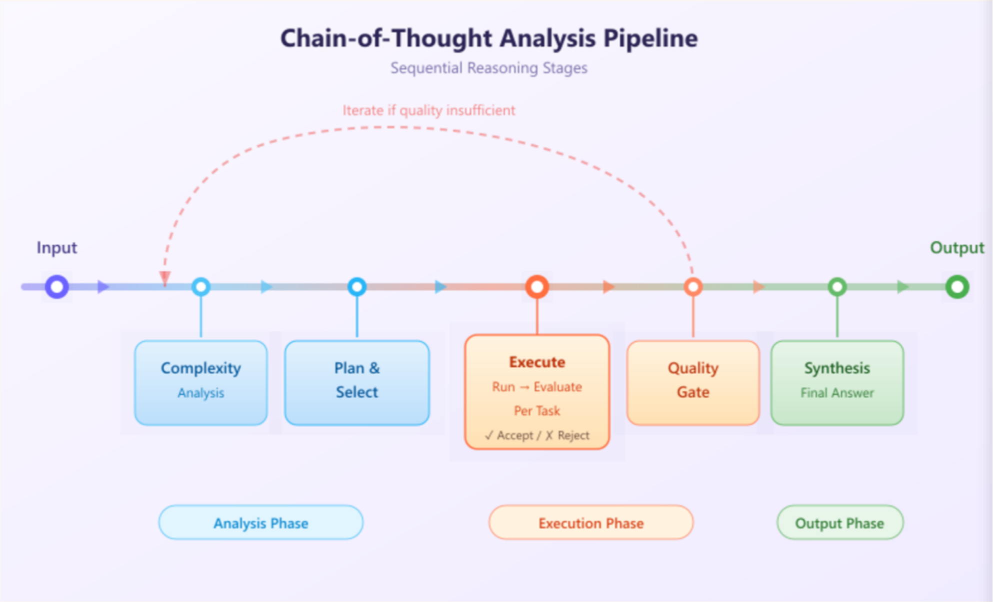
- Organizes multiple stages.
- Passes each output as context into the next.

Why is it intelligent?

- Can change course based on prior results.
- Responds to what it discovers.

Structured Reasoning in Action

mirabella-ai.com — Live chain-of-thought analysis system



Small models, big reasoning

Structured steps let consumer-grade models produce analysis far beyond native capability.

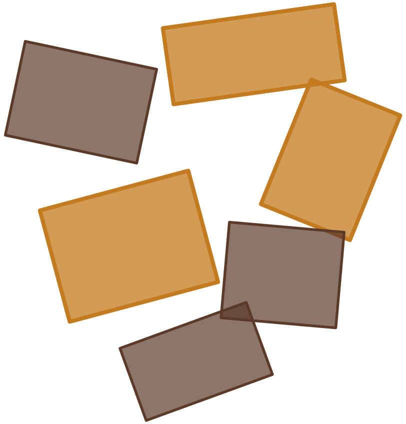
Breaking Down the Problem

Structured outputs enable small models to tackle large problems

The Problem

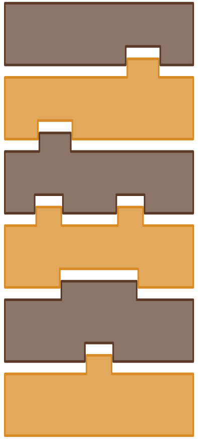
Too large for a small model

Naïve Split



Pieces are random, don't connect

Structured Path



Sequential path — each piece builds on the last

04

Program Recipes

If coding assistants are kitchens, prompts are recipes.

Inside the Recipe

When AI is the kitchen, the recipe is what counts

```
Prompt: Create a program called "ER Triage Structurer"

Step 1: Ingest ER Transcripts
- Read transcript files from ./data/er_transcripts/
- Each file is one raw ER intake transcript.

Step 2: Extract with Structured Output
- Send each transcript to an LLM with structured output.
- Schema: "Triage"
  └─ calculations: list of ············ «RiskCalc»
       ├─ clusters: list of ·········· «SymptomCluster»
       |     ├─ symptoms: list of ·· «SymptomRead»
       |     |     ├─ clinical (text) — medical term
       |     |     └─ severity (int) — -5 to +5
       |     └─ acute (bool) — immediate intervention?
       └─ risk (decimal) — 0.0=harmless … 1.0=critical
  └─ judgment: one of [surgery | observation | discharge | …]

Step 3: Validate and export structured JSON per patient
- Flag any transcript where acute=true or risk ≥ 0.7.
```

Apps by Instruction

Coding agents can turn prompts into full applications:

- Reads the spec you wrote
- Wires it into LLM API calls
- Builds the full application

The End of Language Loyalty?

When AI is the kitchen, the recipe is what counts

THE PROMPT

Your intent, expressed precisely

=

THE RECIPE

"Build a concurrent web scraper with graceful rate limiting"

→

AI CODING ASSISTANT

Claude Code — fluent in every language

=

THE KITCHEN

A master chef who has internalized every culinary tradition

→

PROGRAMMING LANGUAGE

Python, Rust, Go, JS...

=

CULINARY TRADITION

French, Japanese, Italian — different forms, same nourishment

Thank You

Questions & Discussion

Matt Hergott
mirabella-ai.com
contact@mirabella-ai.com